Лабораторная работа №4. Параллельное умножение матриц с использованием OpenMP

Марданов Самир, группа 11-401

Описание работы. Реализованы пять вариантов вычисления $C = AB$ для матриц $N \times N$ типа `double`: `ijk_seq`, `ijk_omp`, `ikj_seq`, `ikj_omp` (`#pragma omp parallel for` по строкам i) и `blocked_omp` (блоки C размера $b \times b$ распределяются между потоками через `collapse(2)`, каждый блок C обновляет только один поток). Матрицы выделяются через `malloc` и заполняются детерминированным генератором; перед каждым запуском C обнуляется, измеряется только умножение (`omp_get_wtime()`), берётся медиана 5 запусков. Все варианты сверялись с `ijk_seq` (допуск 10^{-9}) — проверка пройдена везде. Сборка `gcc-16 -O3 -fopenmp` (GCC 16.2), Apple M1 Pro (6 производительных + 2 энергоэффективных ядра, 16 ГБ), macOS. Запуск: `OMP_NUM_THREADS=p ./matmul N b [reps]`.

Эксперимент 1. 8 потоков, $b = 32$, время, с

N	ijk_seq	ijk_omp	ikj_seq	ikj_omp	blocked_omp
128	0.00352	0.00063	0.00064	0.00020	0.00024
256	0.0157	0.00314	0.00317	0.00072	0.00097
512	0.1451	0.0275	0.0234	0.00506	0.00885
1024	1.4015	0.3035	0.1866	0.0322	0.0729

Эксперимент 2. $N = 1024$, $b = 32$ ($T_{\text{ijk_seq}} = 1.357$ с, $T_{\text{ikj_seq}} = 0.1844$ с)

Потоки	ijk_omp, с	S	E	ikj_omp, с	S	E	blocked_omp, с
1	1.3568	1.00	1.00	0.1839	1.00	1.00	0.4269
2	0.7902	1.72	0.86	0.0947	1.95	0.97	0.2225
4	0.4254	3.19	0.80	0.0485	3.80	0.95	0.1163
8	0.2754	4.93	0.62	0.0316	5.84	0.73	0.0695
16	0.2788	4.87	0.30	0.0349	5.29	0.33	0.0758

Эксперимент 3. $N = 1024$, 8 потоков

b	blocked_omp, с	$T_{\text{ikj_seq}}/T_{\text{blocked}}$
16	0.1094	1.68
32	0.0705	2.61
64	0.0438	4.21
128	0.0425	4.34

Вывод. Главный фактор производительности — порядок циклов. Последовательный `ikj` быстрее `ijk` в 5–7.5 раз (при $N = 1024$: 0.19 с против 1.4 с): во внутреннем цикле `ijk` матрица B читается по столбцу с шагом $N \cdot 8$ байт и каждое обращение попадает в новую кэш-линию, а в `ikj` строки B и C проходятся последовательно, что даёт полное использование кэш-линий и векторизацию. Поэтому даже `ijk_omp` на 8 потоках (0.28 с) медленнее однопоточного `ikj_seq`. Распараллеливание по строкам C не требует синхронизации, и до 4 потоков ускорение близко к линейному ($E \approx 0.95$ у `ikj`); у `ijk` эффективность ниже (0.80), т.к. потоки конкурируют за пропускную способность памяти. Максимум достигнут на 8 потоках (5.8 для `ikj`, 4.9 для `ijk`): из 8 ядер только 6 производительных, а 2 энергоэффективных заметно слабее и задерживают статическое распределение итераций. На 16 потоках ускорение уже не растёт и даже падает — ядер всего 8, лишние потоки добавляют только переключения. Блочная версия на этой машине уступает `ikj_omp`: при $N \leq 1024$ матрицы (по 8 МБ) почти помещаются в L2 (12 МБ на кластер) и системный кэш, поэтому выигрыш от блочной локальности мал, а при малом блоке короткий внутренний цикл хуже векторизуется и увеличивает накладные расходы. С ростом блока время падает с 0.109 с ($b = 16$) до 0.043 с ($b = 64$–128), приближаясь к `ikj_omp`; оптимум около $b = 64$, когда три блока (96 КБ) ещё помещаются в кэш ядра. Таким образом, производительность определяется не только числом потоков, но в первую очередь локальностью доступа к памяти.